



UNDO/REDO TEXT EDITOR

Using Stack Data Structure in C++
Implementation & Analysis



ABSTRACT

- A console-based text editor demonstrating Undo and Redo functionality.
- Uses two stacks to maintain history.
- Efficient state management using STL.



INTRODUCTION

- Importance of Undo/Redo in software.
- Stack: LIFO (Last-In First-Out) principle.
- Snapshot-based approach for text recovery.

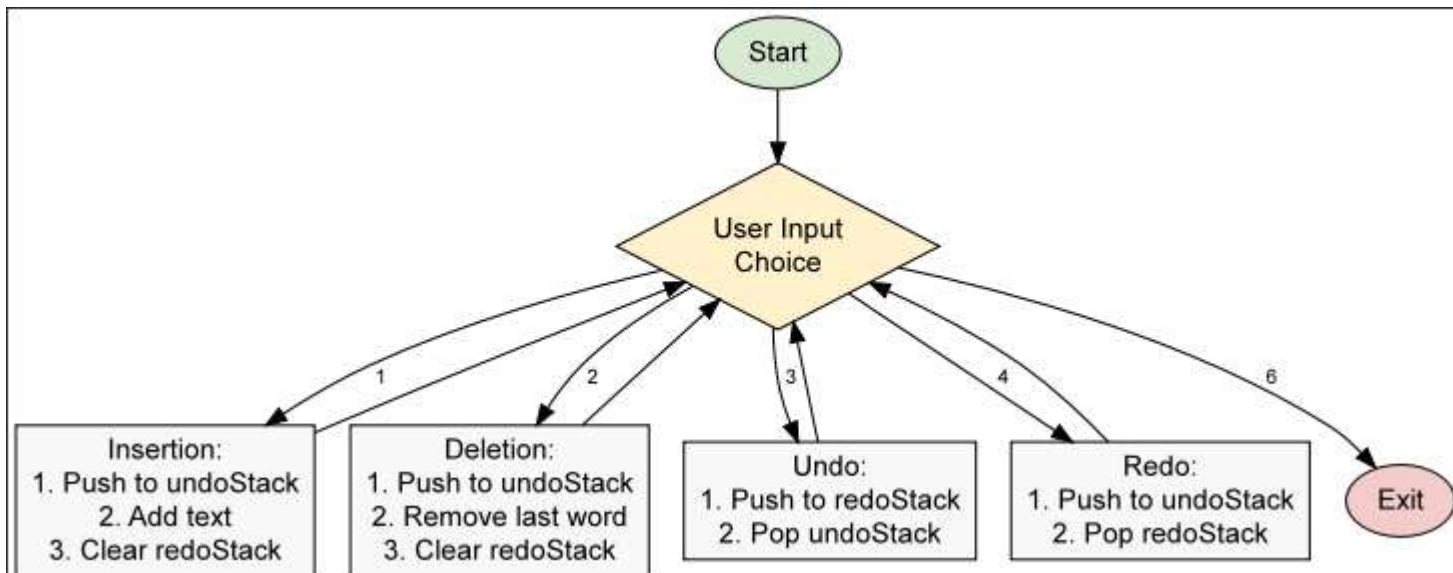


ALGORITHM

- 1. Insertion: Save state to UndoStack, add text, clear RedoStack.
- 2. Deletion: Save state to UndoStack, remove last word, clear RedoStack.
- 3. Undo: Push current to RedoStack, pop from UndoStack.
- 4. Redo: Push current to UndoStack, pop from RedoStack.

FLOWCHART LOGIC

- - Start -> User Input Menu
- - Action -> Check Stack Capacity -> Update Current State
- - Undo/Redo -> Swap states between Stacks
- - Loop until Exit signal.



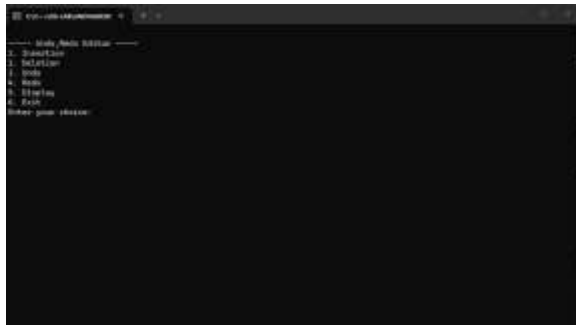


METHODOLOGY

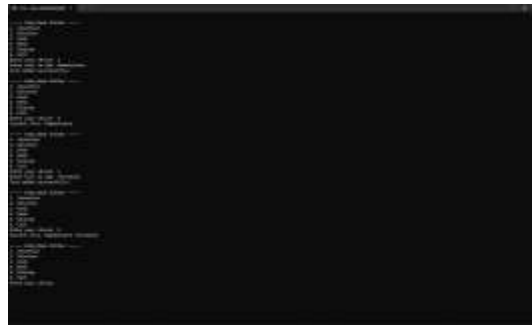
- - Language: C++
- - Library: `<stack>`, `<string>`, `<iostream>`
- - Core Logic: Menu-driven interaction
- - Data Type: `std::stack<std::string>`

RESULT SCREENSHOTS

- 1. Main Menu
- 2. Insertion Test
- 3. Undo/Redo Flow



```
----- Main Menu -----  
1. Insertion  
2. Selection  
3. Undo  
4. Redo  
5. Exit  
Enter your choice:
```



```
----- Insertion Test -----  
1. Insertion  
2. Selection  
3. Undo  
4. Redo  
5. Exit  
Enter your choice:
```



```
----- Undo/Redo Flow -----  
1. Insertion  
2. Selection  
3. Undo  
4. Redo  
5. Exit  
Enter your choice:
```



CONCLUSION

- Successfully implemented basic text history management.
- Demonstrated real-world application of Stacks.
- Simple and efficient CLI interface.

THANK YOU!

- **GROUP DETAILS :**

1. **NAME :** YELUBANDI HARI DURGA RAMAKRISHNA
ROLL NO : 25A31A05LI **SECTION :** CSE F
2. **NAME :** ADABALA DHARMA VEERA SRIKRISHNA KARTHIK
ROLL NO : 25A31A05KB **SECTION :** CSE F
3. **NAME :** ANGADA SAI VIGNESH
ROLL NO : 25A31A05KD **SECTION :** CSE F
4. **NAME :** CHOLLANGI LAKSHITHA
ROLL NO : 25A31A05JB **SECTION :** CSE F